SnowPole-VehicleLoc: An Open-Source Software Framework for Vehicle Localization Using Georeferenced Snow Poles

Durga Prasad Bavirisetti[a,*]

[a] *Department of Computer Science, University of Gävle, Sweden*

Abstract

Reliable vehicle localization remains challenging in GNSS-limited and GNSS-denied environments, particularly under harsh Nordic winter conditions where visual cues are frequently obscured by snow. This paper presents **SnowPole-VehicleLoc**, an open-source, end-to-end software framework for vehicle localization using **georeferenced snow poles** and **LiDAR–GNSS data fusion**. Snow poles are treated as stable, machine-perceivable roadside infrastructure landmarks that remain detectable under snow-covered conditions using LiDAR-based perception.

SnowPole-VehicleLoc is motivated by the need for a reproducible and software-oriented realization of infrastructure-assisted vehicle localization. While the underlying localization methodology has been presented previously, the present work focuses on the software implementation of the approach. In particular, it provides a complete and modular realization of the localization pipeline, together with dynamic visualization and demonstration tools, as well as the datasets and source code required to develop, reproduce, and extend the system.

The framework integrates LiDAR-based snow pole detection, GNSS–LiDAR fusion for landmark anchoring, and incremental LiDAR-based vehicle navigation to enable continuous vehicle pose estimation under varying levels of GNSS availability. SnowPole-VehicleLoc is validated using real-world LiDAR and GNSS data collected on Norwegian highways under Nordic winter conditions.

Keywords: Vehicle localization, Snow poles, GNSS–LiDAR fusion, Infrastructure landmarks, Nordic winter conditions, Incremental navigation, ROS bag processing

*Corresponding author
Email address: `durga.prasad.bavirisetti@hig.se` (Durga Prasad Bavirisetti)

Metadata

The ancillary data presented in Table 1 describes the specific sub-version of the codebase used in this paper. The right column reports metadata corresponding to the current release of the *SnowPole-VehicleLoc* software, while the left column follows the standard SoftwareX metadata schema.

Nr.	**Code metadata description**	**Metadata**
C1	Current code version	v1.0
C2	Permanent link to code/repository used for this code version	`https://github.com/bdps1989/Snow-pole-based-vehicle-localization`
C3	Permanent link to Reproducible Capsule	Not provided (optional)
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python; Conda; PyTorch; OpenCV; ROS (ROS bag utilities); QGIS (external visualization)
C7	Compilation requirements, operating environments & dependencies	Linux (tested on Ubuntu); Python 3.9.18; Conda environment provided via `environment.yml`.
C8	If available Link to developer documentation/manual	`https://github.com/bdps1989/Snow-pole-based-vehicle-localization`
C9	Support email for questions	durga.prasad.bavirisetti@hig.se

Table 1: Code metadata (mandatory)

1. Motivation and significance

Reliable vehicle localization is a fundamental requirement for autonomous driving and advanced driver assistance systems. However, maintaining accurate vehicle pose estimates remains challenging in scenarios where GNSS signals are degraded, intermittently unavailable, or completely denied, often resulting in accumulated odometry drift and long-term localization errors [1, 8]. These challenges are particularly critical for safety-critical applications that require continuous and reliable position estimates over extended trajectories.

In real-world road environments, localization performance is further degraded by prolonged GNSS outages and the limited availability of persistent environmental landmarks that can be consistently perceived across time and sensing conditions. This limitation becomes especially pronounced in long-distance driving and infrastructure-sparse regions, where absolute positioning updates are infrequent and localization systems must rely heavily on incremental motion estimation, increasing susceptibility to drift [8].

The problem is amplified in Nordic winter conditions, where snow-covered roads, reduced visibility, and harsh weather significantly limit the effectiveness of vision-based localization cues. At the same time, terrain-induced effects, vegetation, and sparse roadside infrastructure further reduce GNSS reliability, increasing the risk of unbounded localization drift if no alternative absolute references are available [1].

Snow poles represent a distinctive class of roadside infrastructure that is specifically designed to remain observable under snowfall and adverse weather conditions. When reliably detected using LiDAR-based perception and associated with known map-level coordinates, snow poles provide spatially stable landmarks that can be exploited for vehicle pose correction during GNSS outages [2, 3]. While prior works have demonstrated the feasibility of snow pole detection and geo-localization [3, 2, 4], their effective use for continuous vehicle localization requires tight integration with motion estimation [8] and sensor fusion rather than isolated landmark positioning.

SnowPole-VehicleLoc is motivated by this gap and presents an end-to-end, reproducible software framework for vehicle localization using georeferenced snow poles. While [1] primarily addresses the theoretical and practical aspects of vehicle localization based on georeferenced snow poles, the present work emphasizes the software realization of the approach. In particular, it provides a complete and modular implementation of the localization pipeline, together with dynamic visualization and demonstration tools, as well as the datasets and source code required to develop, reproduce, and extend the system.

The proposed software integrates LiDAR-based snow pole detection [2], GNSS–LiDAR fusion for landmark anchoring [3], and incremental LiDAR-based vehicle navigation using point cloud registration [8]. By combining these components within a unified pipeline, the framework enables continuous and robust vehicle pose estimation under varying levels of GNSS availability.

By treating snow poles as localization anchors within an integrated localization framework, SnowPole-VehicleLoc supports effective drift mitigation and trajectory stabilization in winter-degraded and infrastructure-sparse environments. This approach extends beyond standalone geo-localization and enables robust vehicle localization in conditions where conventional GNSS- or vision-based methods often fail [1].

2. Software architecture

The *SnowPole-VehicleLoc* repository follows a modular software design, clearly separating the core vehicle localization pipeline from supporting components such as coordinate transformation utilities, ROS bag processing and visualization tools, model assets, configuration files, and reproducibility resources. This organization improves code clarity and maintainability, while facilitating systematic experimentation and straightforward integration into larger vehicle localization and navigation systems.

2.1. Repository organization

The SnowPole-VehicleLoc repository is structured to clearly separate the core vehicle localization logic from auxiliary utilities for data processing, visualization, evaluation, and reproducibility. This modular organization supports extensibility, debugging, and reuse in related localization and navigation systems.

```
Snow-pole-based-vehicle-localization/

 snowpole_based_vehicle_localization.py
   Main pipeline for snow-pole-based vehicle localization


 snowpole_based_vehicle_localization_GNSS_percentage.py
   Variant of the main pipeline for evaluating localization performance under
   varying GNSS availability levels

 geoloc_utils.py
   Utility functions for coordinate transformations, geodesic calculations,
   data association, and localization evaluation

 rosbag_utils/
   ROS bag processing and visualization utilities

    extract_rosbag_topics.py
      Extraction of required LiDAR and GNSS topics from ROS bag files

    gnss_and_groundtruth_snowpole_visualization.py
      Visualization of GNSS trajectories and ground-truth snow pole locations

    lidar_image_visualization.py
```

```
      Visualization of LiDAR-derived images

    range_image_to_pointcloud_visualization.py
      Conversion and visualization of point clouds from LiDAR range images

    realpointcloud_visualization.py
       Visualization of raw LiDAR point clouds

 model/
   Pretrained snow pole detection models and related configuration files

 Groundtruth_pole_location_at_test_site_E39_Hemnekjølen.csv
   Georeferenced ground-truth snow pole locations for localization and
   evaluation
 Trip068.json
   Sample trip metadata and configuration parameters

 incremental_navigation_results.csv
   Example output containing incremental vehicle localization estimates

 environment.yml
   Conda environment specification ensuring reproducibility
```

2.2. *snowpole_based_vehicle_localization.py*

This script implements the end-to-end snow-pole-based vehicle localization pipeline. It integrates LiDAR-based snow pole detection, snow pole geo-localization, GNSS–LiDAR fusion, data association, and incremental LiDAR-based vehicle navigation to estimate a continuous vehicle trajectory. The script serves as the primary entry point for executing the complete localization workflow using sensor data exported from ROS bag recordings.

2.3. *snowpole_based_vehicle_localization_GNSS_percentage.py*

This script extends the main localization pipeline to evaluate robustness under varying levels of GNSS availability. By selectively enabling or disabling GNSS updates according to a specified percentage, it enables controlled experiments under partial GNSS availability and complete GNSS denial scenarios. The outputs support quantitative analysis of localization accuracy as a function of GNSS availability and facilitate comparison with LiDAR-only navigation baselines.

2.4. `geoloc_utils.py`

This module contains reusable utility functions required by the localization pipeline. It includes routines for coordinate transformations between geographic and local metric frames, geodesic distance calculations, association of observed snow poles with georeferenced landmarks, and computation of localization error statistics. Isolating these operations improves code clarity, reuse, and maintainability.

2.5. `rosbag_utils/`

The `rosbag_utils/` directory provides scripts for extracting, inspecting, and visualizing sensor data stored in ROS bag files. These utilities support reproducible preprocessing and qualitative validation of LiDAR-derived images, raw point clouds, and GNSS trajectories prior to running the vehicle localization pipeline.

2.5.1. `extract_rosbag_topics.py`

This script extracts the required LiDAR and GNSS topics from raw ROS bag files and stores them in a compact, experiment-ready format. It reduces processing overhead by removing unrelated sensor streams and ensures consistent data inputs across experiments.

2.5.2. `gnss_and_groundtruth_snowpole_visualization.py`

This script visualizes the vehicle GNSS trajectory together with the measured ground-truth snow pole locations. It is primarily used for qualitative inspection of spatial alignment and verification of the test site geometry.

2.5.3. `lidar_image_visualization.py`

This script provides visualization tools for LiDAR-derived images (e.g., signal or reflectance representations). It supports inspection of image quality and snow pole visibility under winter conditions.

2.5.4. `range_image_to_pointcloud_visualization.py`

This script converts LiDAR range images back into 3D point clouds and visualizes the reconstructed geometry. It enables verification of the mapping between image-space detections and corresponding 3D spatial measurements.

2.5.5. `realpointcloud_visualization.py`

This script visualizes raw LiDAR point clouds directly from ROS bag data. It is useful for inspecting original sensor measurements, validating data quality, and analyzing failure cases caused by occlusion or adverse weather.

2.6. `model/`

The `model/` directory contains pretrained snow pole detection models and related configuration files used for inference within the localization pipeline. These models are applied to LiDAR-derived images to generate snow pole detections that are subsequently mapped to spatial observations.

2.7. `Groundtruth_pole_location_at_test_site_E39_Hemnekjølen.csv`

This CSV file contains manually measured, georeferenced snow pole locations at the E39–Hemnekjølen test site. It serves as the reference for quantitative evaluation, enabling computation of localization errors and performance statistics.

2.8. `Trip068.json`

This JSON file provides sample trip metadata and configuration parameters, including dataset identifiers and file path settings. It supports reproducible experimentation by ensuring that dataset-specific parameters are consistently managed across runs.

2.9. `incremental_navigation_results.csv`

This file contains example outputs from the incremental vehicle localization process, including estimated vehicle positions over time. The results are generated using incremental LiDAR-based navigation based on the FastReg point cloud registration method [8]. The file illustrates the format of localization outputs produced by the framework and can be used for post-processing, visualization, and quantitative evaluation.

2.10. `environment.yml`

This Conda environment specification defines all required software dependencies and the Python version (Python 3.9.18) used to run the framework. Providing a controlled environment ensures reproducibility of experiments across different systems and simplifies setup for new users.

3. Software functionalities

SnowPole-VehicleLoc provides a set of core functionalities for infrastructure-assisted vehicle localization in GNSS-limited and GNSS-denied environments. The main capabilities of the framework are summarized as follows:

- **Snow pole detection and landmark extraction** from LiDAR-derived signal images using a YOLOv5s-based detector [4, 9] trained on 1,954 annotated images.

- **Snow pole geo-localization** by estimating pole distance and bearing from LiDAR observations and fusing these measurements with GNSS data to obtain map-level coordinates aligned with ground-truth pole locations.
- **Incremental vehicle localization** using LiDAR-based point cloud registration to propagate vehicle motion during partial or complete GNSS outages.
- **GNSS–LiDAR fusion** that combines GNSS-based absolute positioning with LiDAR-based incremental navigation, while using georeferenced snow pole landmarks to mitigate accumulated drift.
- **GNSS availability analysis** through a dedicated evaluation script that measures localization accuracy under varying percentages of GNSS availability and denial.
- **ROS bag processing and visualization** utilities for LiDAR-derived images, point clouds, GNSS trajectories, and ground-truth snow pole locations.
- **Reproducibility support** via a validated Conda environment (`environment.yml`) tested with Python 3.9.18.

3.1. GNSS–LiDAR fusion and localization workflow

The localization workflow implemented in SnowPole-VehicleLoc consists of the following stages:

(1) Snow pole detection. Snow poles are detected from LiDAR-derived signal images using a YOLOv5s-based CNN. Each detected bounding box is mapped to the corresponding LiDAR range image to retrieve distance and angular information relative to the vehicle, forming landmark observations for localization.

(2) Snow pole geo-localization. Relative pole measurements are fused with GNSS positioning to compute absolute pole geolocations in map coordinates, aligned with ground-truth snow pole locations. This step establishes a stable landmark reference for vehicle pose correction.

(3) Incremental vehicle localization. Vehicle motion is estimated by combining GNSS updates (when available) with incremental LiDAR-based navigation. During GNSS outages, consecutive LiDAR frames are aligned using point cloud registration; the current implementation employs FastReg due to its balance between accuracy and computational efficiency.

(4) Drift mitigation using snow poles. Detected snow poles are used as correction landmarks by registering locally observed pole point clouds with their georeferenced counterparts, reducing odometry drift during prolonged GNSS outages.

(5) GNSS availability evaluation. Localization performance is evaluated under different GNSS availability levels by controlling the proportion of GNSS updates used during navigation. This analysis quantifies robustness and highlights the contribution of snow pole landmarks when GNSS information is sparse or unavailable.

4. Illustrative examples

4.1. Data and experimental setting

The illustrative examples for **SnowPole-VehicleLoc** are based on real-world LiDAR and GNSS sensor data collected along the E39–Hemnekjølen test site in Norway (approximately 4.2 km). The route spans mountainous terrain, forested segments, and open road sections, providing a challenging setting for vehicle localization under GNSS-limited conditions and harsh Nordic winter environments.

The dataset includes high-resolution LiDAR measurements acquired using a 128-channel Ouster OS2-128 sensor, continuous GNSS data from dual receivers, and manually surveyed georeferenced snow pole locations. These components enable end-to-end evaluation of vehicle localization performance, including incremental navigation, landmark-based pose correction, and robustness under partial or complete GNSS outages.

Due to data volume constraints, the repository links to publicly available datasets: (i) Snow pole detection datasets hosted on Mendeley Data [5, 6], and (ii) ROS bag recordings containing LiDAR and GNSS data hosted on Kaggle [7].

4.2. Reproducibility and installation

To ensure reproducibility of the vehicle localization experiments, a Conda environment specification is provided with the repository. The framework has been tested using Python 3.9.18.

```
conda create -n polegeo python=3.9.18 -y
conda activate polegeo
conda env update -f environment.yml --prune
python --version
```

4.3. Example workflow

A typical workflow for running the SnowPole-VehicleLoc framework is as follows:

1. Download a ROS bag file from the Kaggle dataset [7] and place it inside the repository directory (or update file paths in the configuration scripts).
2. Use the scripts under `rosbag_utils/` to inspect LiDAR-derived images, raw point clouds, and GNSS trajectories, and to verify data integrity prior to localization.
3. Run `snowpole_based_vehicle_localization.py` to estimate the vehicle trajectory using GNSS–LiDAR fusion, incremental LiDAR-based navigation, and snow pole landmark corrections.
4. Optionally execute `snowpole_based_vehicle_localization_GNSS_percentage.py` to evaluate localization performance under different GNSS availability percentages.
5. Analyze and visualize the resulting vehicle trajectories and error statistics using the generated output files.

4.4. Demo visualization

Running the main pipeline script `snowpole_based_vehicle_localization.py` produces a dynamic visualization of the snow-pole-based vehicle localization process. The visualization is updated incrementally as the vehicle progresses along the route, displaying the estimated vehicle trajectory together with localization updates derived from GNSS measurements, LiDAR-based incremental navigation, and snow pole landmark corrections.

A demonstration video of the SnowPole-VehicleLoc framework is provided to illustrate the complete vehicle localization workflow on real-world data. The video shows the temporal evolution of the estimated vehicle trajectory, demonstrating how incremental LiDAR-based navigation, georeferenced snow pole landmarks, and GNSS updates are integrated to maintain localization continuity and stabilize the trajectory under GNSS-denied conditions. The dynamic visualization enables qualitative assessment of localization behavior, and drift mitigation that are difficult to capture using static figures alone.

For clarity and ease of interpretation, the video is intended to be played at **2× speed**, which provides a smoother overview of vehicle motion and localization updates along the route. The demonstration video is publicly available at: `https://www.youtube.com/watch?v=MNGnOWTT25Q`.

A representative screenshot of the dynamic visualization generated by the SnowPole-VehicleLoc framework is shown in Figure 1. The figure illustrates the estimated vehicle trajectory together with snow pole–based localization updates during GNSS-limited operation. For a more detailed analysis of localization performance and extended experimental results, the reader is referred to [1]. The SnowPole-VehicleLoc software framework, including source code and documentation, is publicly available via the repository listed in Table 1.

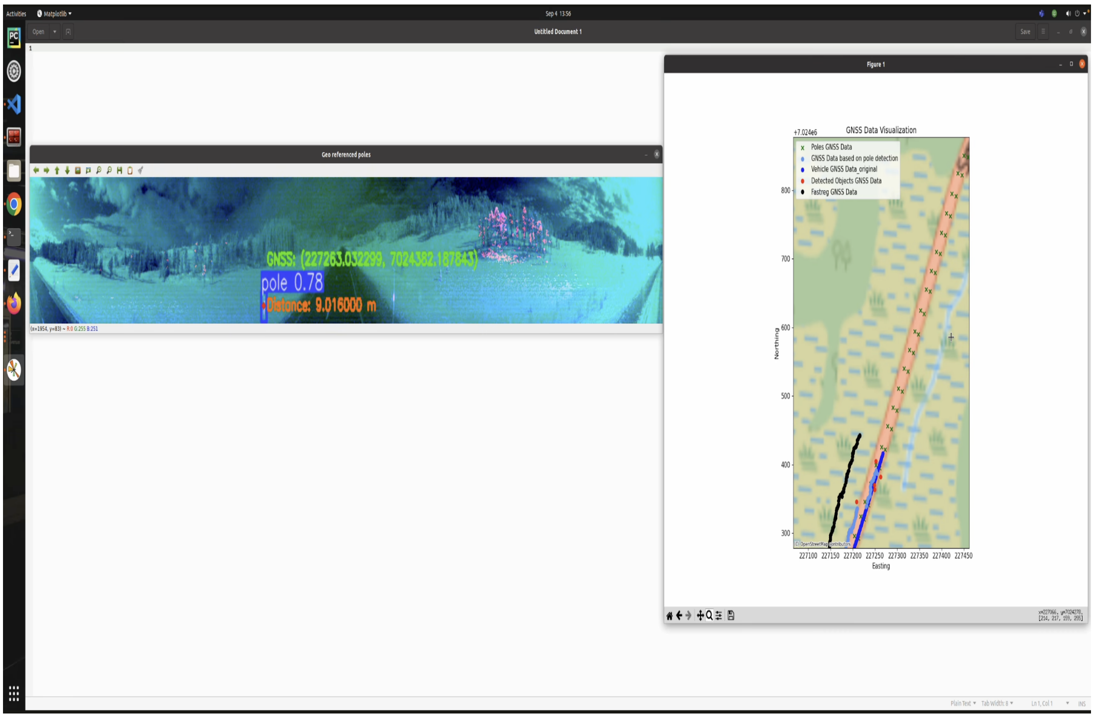

Figure 1: Screenshot of the dynamic snow-pole-based vehicle localization visualization produced by running `snowpole_based_vehicle_localization.py`. The figure shows the estimated vehicle trajectory together with localization updates obtained from LiDAR-based incremental navigation, GNSS measurements, and georeferenced snow pole landmarks.

5. Impact

SnowPole-VehicleLoc facilitates reproducible research and practical development of infrastructure-assisted vehicle localization in winter-degraded and GNSS-limited environments. By providing a complete, open-source implementation of an end-to-end localization pipeline, the framework enables:

- **Exploration of machine-perceivable infrastructure** as reliable localization anchors in conditions where conventional visual and GNSS cues are unreliable or unavailable.

- **Systematic benchmarking** of end-to-end vehicle localization pipelines under real-world Nordic winter conditions and controlled GNSS availability scenarios.
- **Quantitative evaluation of drift mitigation**, allowing researchers to analyze the impact of snow pole landmarks on incremental navigation accuracy during prolonged GNSS outages.
- **Reproducible experimental workflows** for ROS bag–based analysis, sensor visualization, and GIS-supported validation of localization results.

By releasing the software together with pretrained models, dataset references, and environment specifications under open licenses, SnowPole-VehicleLoc lowers the barrier to replication, comparative evaluation, and extension to additional infrastructure landmarks, sensing modalities, and localization backends.

6. Conclusions

This paper presented **SnowPole-VehicleLoc**, an open-source, end-to-end software framework for vehicle localization in GNSS-limited and GNSS-denied environments, with a particular focus on Nordic winter conditions. The framework combines LiDAR-based snow pole detection, GNSS–LiDAR–assisted snow pole geo-localization, incremental LiDAR-based vehicle navigation, and landmark-based drift mitigation within a unified and reproducible pipeline. By providing a modular implementation, evaluation tools, and publicly available datasets and source code, SnowPole-VehicleLoc offers a practical foundation for research and development on infrastructure-assisted vehicle localization under winter-degraded sensing conditions.

Acknowledgements

This research was conducted within the project *"Machine Sensible Infrastructure under Nordic Conditions"* (Project No. 333875). The author gratefully acknowledges the members of the Visual Computing group at the Department of Computer Science, NTNU, and the Mobility Department at SINTEF Community, Trondheim, for their valuable discussions, technical insights, and support throughout this work.

References

[1] D. P. Bavirisetti, G. E. Berget, G. H. Kiss, P. Arnesen, H. Seter, and F. Lindseth, Vehicle localization framework using georeferenced snow poles and LiDAR in GNSS-limited environments under Nordic conditions, *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[2] D. P. Bavirisetti, G. H. Kiss, P. Arnesen, H. Seter, S. Tabassum, and F. Lindseth, SnowPole detection: A comprehensive dataset for detection and localization using LiDAR imaging in Nordic winter conditions, *Data in Brief* 59 (2025) 111403.

[3] D. P. Bavirisetti, G. H. Kiss, and F. Lindseth, A pole detection and geospatial localization framework using LiDAR–GNSS data fusion, in: *Proceedings of the 27th International Conference on Information Fusion (FUSION)*, IEEE, 2024, pp. 1–8.

[4] D. P. Bavirisetti, Extended evaluation of SnowPole detection for machine-perceivable infrastructure for Nordic winter conditions: A comparative study of object detection models, SSRN preprint 5386946, 2025.

[5] D. P. Bavirisetti, G. H. Kiss, P. Arnesen, H. Seter, S. Tabassum, and F. Lindseth, SnowPole Detection: A Comprehensive Dataset for Detection and Localization Using LiDAR Imaging in Nordic Winter Conditions, Mendeley Data, Version 2, 2024. `https://doi.org/10.17632/tt6rbx7s3h.2`.

[6] D. P. Bavirisetti, M. Rafiq, G. H. Kiss, and F. Lindseth, Extended Evaluation of SnowPole Detection for Machine-Perceivable Infrastructure for Nordic Winter Conditions: A Comparative Study of Object Detection Models, Mendeley Data, Version 3, 2025. `https://doi.org/10.17632/tt6rbx7s3h.3`.

[7] D. P. Bavirisetti, G. H. Kiss, F. Lindseth, P. Arnesen, and H. Seter, Data for the snow-pole-based vehicle localization, Kaggle Dataset, 2025. `https://doi.org/10.34740/KAGGLE/DSV/14311103`.

[8] E. Arnold, S. Mozaffari, and M. Dianati, Fast and robust registration of partially overlapping point clouds, *IEEE Robotics and Automation Letters* 7(2) (2021) 1502–1509.

[9] G. Jocher, YOLOv5 by Ultralytics (version 7.0) [Computer software], 2020.